Redis Architecture & Implementation Guide — Enterprise Microservices Reference

Redis for Caching, Distributed Locking, Rate Limiting & Real-Time Systems

Version: 2.0 **Date:** March 2026 **Classification:** Internal — Architecture Reference **Author:** Senior Distributed Systems Architect

---

Table of Contents

1. Executive Summary
2. Redis Overview
3. Redis Internal Architecture
4. Microservices Integration Strategy
5. Step-by-Step Implementation Guide
6. Code Walkthrough
7. Caching Design Patterns
8. Distributed Systems Challenges & Solutions
9. Performance Tuning Guide
10. Security & Monitoring
11. Interview Preparation Questions
12. Future Scalability Considerations

---

1. Executive Summary

1.1 System Overview

The Real-Time Order Processing System uses Redis as a **multi-purpose in-memory data store** across 3 services, serving as:

1. **Application Cache** (Product Service) — Eliminates repeated MySQL queries using `@Cacheable` / `@CachePut` / `@CacheEvict` with JSON serialization and TTL-based expiration
2. **Distributed Lock Manager** (Inventory Service) — Prevents race conditions in stock reservation using SETNX with UUID-based lock ownership and 30-second TTL auto-expiry
3. **Rate Limiter Backend** (API Gateway) — Powers Resilience4j rate limiting to prevent API abuse

1.2 Redis Usage Matrix

```
+--------------------+-------------------+--------------------+-------------------+
| Service            | Redis Purpose     | Implementation     | Key Pattern       |
+--------------------+-------------------+--------------------+-------------------+
| Product Service    | Cache-Aside       | RedisCacheManager  | product::1        |
| (port 8083)        | caching           | @Cacheable/Evict   | products::all     |
|                    |                   | JSON serializer    |                   |
+--------------------+-------------------+--------------------+-------------------+
| Inventory Service  | Distributed       | StringRedisTemp.   | inventory:lock:   |
| (port 8085)        | locking (SETNX)   | setIfAbsent()      | {productId}       |
|                    |                   | UUID lock value    |                   |
+--------------------+-------------------+--------------------+-------------------+
| API Gateway        | Rate limiting     | Resilience4j +     | (internal)        |
```

```
| (port 8080)        | backend          | Redis store       |                   |
+--------------------+------------------+-------------------+-------------------+
```

1.3 Key Architecture Decisions

#	Decision	Rationale
1	Redis over Memcached	Rich data structures, persistence, pub/sub, distributed locks
2	Cache-Aside over Write-Through	Application controls cache population; simpler invalidation
3	JSON serialization over JDK	Human-readable cache entries; cross-language compatible
4	SETNX over Redisson/Redlock	Simple, lightweight; sufficient for single-node Redis
5	UUID lock values	Prevents accidental release of another thread's lock
6	30s lock TTL	Covers worst-case processing time; auto-releases on crash
7	Two-layer concurrency	Redis lock + JPA optimistic locking = defense in depth
8	Per-cache TTL strategy	Products: 10min (stable), product lists: 5min (more volatile)

1.4 Configuration Summary

```
+--------------------+------------------+------------------+-------------------+
| Parameter          | Product Service  | Inventory Service| API Gateway       |
+--------------------+------------------+------------------+-------------------+
| Redis Host         | localhost        | localhost        | localhost         |
| Redis Port         | 6379             | 6379             | 6379              |
| Cache Type         | redis            | —                | —                 |
| Default TTL        | 10 min           | —                | —                 |
| List Cache TTL     | 5 min            | —                | —                 |
| Lock TTL           | —                | 30 sec           | —                 |
| Null Caching       | Disabled         | —                | —                 |
| Key Serializer     | String           | String           | —                 |
| Value Serializer   | Jackson JSON     | String           | —                 |
| Rate Limit Period  | —                | —                | 100 req/min       |
+--------------------+------------------+------------------+-------------------+
```

---

2. Redis Overview

2.1 What is Redis?

Redis (Remote Dictionary Server) is an **open-source, in-memory data structure store** used as a database, cache, message broker, and streaming engine. It supports multiple data structures including strings, hashes, lists, sets, sorted sets, bitmaps, hyperloglogs, and streams.

Key characteristics:

- **In-memory:** All data stored in RAM (100x–1000x faster than disk-based databases)
- **Single-threaded:** Uses event-loop architecture (no concurrency bugs, no locks needed internally)
- **Persistence-optional:** Can snapshot to disk (RDB) or append-only log (AOF)

- **Rich data types:** Not just key-value; supports complex structures natively
- **Sub-millisecond latency:** Typical GET/SET operations complete in <1ms

2.2 Why Redis in Microservices?

```
PROBLEM 1: Repeated Database Queries
────────────────────────────────────────────

Product Service receives 1000 GET /products/1 requests per second.
Each request queries MySQL: SELECT * FROM products WHERE id = 1

MySQL: ~5ms per query × 1000 = 5000ms of DB time per second
Redis:  ~0.1ms per query × 1000 = 100ms of Redis time per second

Result: 50x FEWER database connections, 50x LOWER latency.

PROBLEM 2: Race Conditions in Distributed Systems
────────────────────────────────────────────

Inventory Service runs 3 instances behind a load balancer.
100 users simultaneously buy the last item (stock = 1).
Without coordination: all 3 instances read stock=1, all reserve → oversold!
Redis SETNX: Only ONE instance can acquire the lock → prevents overselling.

PROBLEM 3: API Abuse / DDoS Protection
────────────────────────────────────────────

API Gateway must limit each user to 100 requests per minute.
Without Redis: Each gateway instance counts independently → user sends
               100 to Instance A and 100 to Instance B = 200 total.
With Redis: Centralized counter → user limited to 100 TOTAL across all instances.
```

2.3 Why Redis Instead of Traditional DB Caching?

```
+──────────────────+────────────────────+────────────────────+
│ Feature          │ MySQL Query Cache  │ Redis              │
+──────────────────+────────────────────+────────────────────+
│ Location         │ Same server as DB  │ Separate service   │
│ Scalability      │ Scales with DB     │ Scales independently│
│ Data structures  │ Rows only          │ Strings, Hashes,   │
│                  │                    │ Lists, Sets, ZSets │
│ TTL control      │ No granular TTL    │ Per-key TTL        │
│ Invalidation     │ Automatic (on write)│ Manual/programmatic│
│ Cross-service    │ Single DB only     │ Shared across all  │
│ sharing          │                    │ microservices      │
│ Persistence      │ Disk-based         │ RAM + optional disk│
│ Latency          │ ~5ms               │ ~0.1ms             │
│ Atomic ops       │ Limited            │ INCR, SETNX, MULTI │
│ Pub/Sub          │ No                 │ Built-in           │
│ Distributed lock │ No                 │ SETNX with TTL     │
+──────────────────+────────────────────+────────────────────+

CONCLUSION: MySQL query cache is limited to a single database server.
```

```
Redis is an independent service that provides sub-millisecond access,
rich data structures, distributed locking, and pub/sub — all in one.
```

2.4 When NOT to Use Redis

```
1. LARGE DATASET (> available RAM):
   Redis stores everything in RAM. If your dataset is 100GB and you have 32GB RAM,
   use a disk-based cache (e.g., RocksDB, Apache Ignite).
   RULE: Redis dataset must fit in available RAM.

2. STRONG CONSISTENCY REQUIRED:
   Redis is eventually consistent in cluster mode (async replication).
   For ACID transactions, use your primary database.
   EXAMPLE: Bank account balances → MySQL transactions, NOT Redis.

3. COMPLEX QUERIES:
   Redis has no SQL, no JOINs, no aggregations.
   For analytical queries → MySQL, PostgreSQL, or Elasticsearch.

4. LONG-TERM STORAGE:
   Redis is optimized for hot data (frequently accessed).
   For archival data → S3, HDFS, or cold storage.

5. SECURITY-SENSITIVE DATA (without proper setup):
   Redis plaintext by default. For PII/PHI, enable TLS + AUTH + ACLs.

6. PROTOTYPING/MVP (single server, low traffic):
   In-process caching (Caffeine, Guava) is simpler and sufficient.
   EVALUATE: Do you need distributed caching? If not, in-process is fine.
```

2.5 Where Redis Fits in This Architecture

```
+──────────────────────────────────────────────────────────────────────────+
│                       CLIENT (Browser/Mobile)                            │
│                                                                          │
│                     ┌──────────────────────────┐                         │
│                     │        API GATEWAY       │                         │
│                     │     (Spring Cloud Gateway)  │                      │
│                     │                          │                         │
│                     │  ┌────────────────────┐  │                         │
│                     │  │ Redis Rate Limiter │──┼──────────┐              │
│                     │  │ (100 req/min/user) │  │          │              │
│                     │  └────────────────────┘  │          │              │
│                     └──────────┬───────────────┘          │              │
│                                │                          │              │
│              ┌─────────────────┼─────────────────┐        │              │
│              │                 │                 │        │              │
│              ▼                 ▼                 ▼        ▼              │
│   ┌──────────────────┐ ┌──────────────┐ ┌─────────────────┐ ┌────────┐  │
│   │ Product Service  │ │ Order Service│ │ Inventory Svc   │ │ REDIS  │  │
│   │                  │ │              │ │                 │ │ Server │  │
```

```
│   │ ┌──────────────┐ │ │             │ │ ┌──────────────┐ │ │        │  │
│   │ │ @Cacheable   │─┼─┼─────────────┼─┼─│              │ │ │ 6379   │  │
│   │ │ @CachePut    │ │ │             │ │ │ SETNX Lock   │─┼─│        │  │
│   │ │ @CacheEvict  │ │ │             │ │ │ (30s TTL)    │ │ │        │  │
│   │ └──────────────┘ │ │             │ │ └──────────────┘ │ │        │  │
│   │        │         │ │             │ │        │         │ │        │  │
│   │        ▼         │ │      ▼      │ │        ▼         │ └────────┘  │
│   │   product_db     │ │  order_db   │ │  inventory_db    │             │
│   │   (MySQL)        │ │  (MySQL)    │ │  (MySQL)         │             │
│   └──────────────────┘ └─────────────┘ └──────────────────┘            │
+──────────────────────────────────────────────────────────────────────────+
```

---

3. Redis Internal Architecture

3.1 Single-Threaded Event Loop

```
WHY SINGLE-THREADED?
  Traditional databases use multi-threading:
    Thread 1: handles client A (locks, context switches, synchronization)
    Thread 2: handles client B (locks, context switches, synchronization)
    → Overhead from locking + context switching → ~10% performance loss

  Redis uses single-threaded event loop (epoll/kqueue):
    Main Thread: handles ALL clients sequentially (no locks, no switches)
    → Zero locking overhead → maximum throughput from a single core

BUT ISN'T SINGLE-THREADED SLOW?
  No! Because:
  1. All data is in RAM → no disk I/O wait
  2. Operations are O(1) or O(log N) → sub-microsecond per command
  3. Network I/O is multiplexed via epoll → handles 100K+ connections
  4. No thread creation/destruction overhead

  Result: Single Redis thread handles ~100,000 operations per second.

REDIS 6.0+ MULTI-THREADING:
  Redis 6.0 introduced I/O threads for network read/write:
    Main Thread: still executes all commands (single-threaded)
    I/O Threads: handle socket read/write in parallel
    → Improves network throughput, NOT command execution
    → Command execution remains atomic and single-threaded

EVENT LOOP DIAGRAM:
  ┌──────────────────────────────────────────────────────┐
  │                    EVENT LOOP                        │
  │                                                      │
  │  ┌──────────┐   ┌──────────┐   ┌──────────┐        │
  │  │ Client A│   │ Client B│   │ Client C│        │
  │  │  GET x  │   │  SET y  │   │  INCR z │        │
  │  └────┬─────┘   └────┬─────┘   └────┬─────┘        │
  │       │              │              │              │
```

```
  │       ▼              ▼              ▼              │
  │  ┌──────────────────────────────────────────┐     │
  │  │      I/O MULTIPLEXER (epoll/kqueue)      │     │
  │  │         Monitors all client sockets      │     │
  │  └──────────────────────┬───────────────────┘     │
  │                         │                          │
  │                         ▼                          │
  │  ┌──────────────────────────────────────────┐     │
  │  │      COMMAND PROCESSOR (single thread)   │     │
  │  │   → Process GET x (return value)         │     │
  │  │   → Process SET y (update hash table)    │     │
  │  │   → Process INCR z (atomic increment)    │     │
  │  └──────────────────────┬───────────────────┘     │
  │                         │                          │
  │                         ▼                          │
  │  ┌──────────────────────────────────────────┐     │
  │  │      RESPONSE WRITER                     │     │
  │  │   → Send response to Client A            │     │
  │  │   → Send response to Client B            │     │
  │  │   → Send response to Client C            │     │
  │  └──────────────────────────────────────────┘     │
  └──────────────────────────────────────────────────┘
```

3.2 Data Structures

```
+──────────────+──────────────────────+──────────────────────────+
| Type         | Use Case             | This Project             |
+──────────────+──────────────────────+──────────────────────────+
| STRING       | Cache values,        | Product cache (JSON string) |
|              | counters, locks      | Distributed lock (SETNX)    |
|              |                      | Rate limit counters         |
+──────────────+──────────────────────+──────────────────────────+
| HASH         | Object storage,      | (available for user         |
|              | user sessions        |  sessions, cart data)       |
+──────────────+──────────────────────+──────────────────────────+
| LIST         | Message queues,      | (available for notification |
|              | activity feeds       |  queue, recent orders)      |
+──────────────+──────────────────────+──────────────────────────+
| SET          | Unique collections,  | (available for tracking     |
|              | tag systems          |  active users)              |
+──────────────+──────────────────────+──────────────────────────+
| SORTED SET   | Leaderboards,        | (available for product      |
| (ZSET)       | priority queues      |  rankings, trending items)  |
+──────────────+──────────────────────+──────────────────────────+
| STREAM       | Event log,           | (alternative to Kafka       |
|              | message broker       |  for lightweight use cases) |
+──────────────+──────────────────────+──────────────────────────+

STRING OPERATIONS (Used in this project):
  SET key value              → Store a value
  GET key                    → Retrieve a value
```

```
  SET key value NX EX 30      → SETNX with 30s expiry (distributed lock)
  INCR key                    → Atomic increment (rate limiting)
  DEL key                     → Delete a key
```

3.3 Persistence (RDB vs AOF)

```
+--------------------+----------------------+----------------------+
| Feature            | RDB (Snapshots)      | AOF (Append-Only)    |
+--------------------+----------------------+----------------------+
| How                | Point-in-time dump   | Logs every write     |
|                    | of entire dataset    | command sequentially |
+--------------------+----------------------+----------------------+
| Speed              | Fast recovery (load  | Slower recovery      |
|                    | binary dump)         | (replay all commands)|
+--------------------+----------------------+----------------------+
| Data Loss          | Minutes (between     | Seconds (based on    |
|                    | snapshots)           | fsync policy)        |
+--------------------+----------------------+----------------------+
| File Size          | Compact (binary)     | Large (text commands)|
+--------------------+----------------------+----------------------+
| CPU Impact         | Periodic fork()      | Continuous logging   |
|                    | for snapshot          | (minimal CPU)        |
+--------------------+----------------------+----------------------+
| Use Case           | Backups, disaster    | High durability      |
|                    | recovery             | requirements         |
+--------------------+----------------------+----------------------+

RECOMMENDATION FOR THIS PROJECT:
  Development: No persistence needed (data is transient cache/locks)
  Production:  RDB every 5 minutes + AOF with everysec fsync
               → Covers both fast recovery and minimal data loss

CONFIGURATION (redis.conf):
  # RDB: save <seconds> <changes>
  save 300 100     # Snapshot every 5 minutes if 100+ keys changed
  save 60 10000    # Snapshot every 1 minute if 10000+ keys changed

  # AOF
  appendonly yes
  appendfsync everysec  # Flush to disk every second (max 1s data loss)
```

3.4 Replication

```
MASTER-REPLICA ARCHITECTURE:

   ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
   │   MASTER     │────▶│  REPLICA 1   │     │  REPLICA 2   │
   │  (read/write)│     │  (read only) │     │  (read only) │
   │              │────▶│              │     │              │
   └──────────────┘     └──────────────┘     └──────────────┘
                   async replication          async replication
```

```
WHY REPLICATION?
  1. HIGH AVAILABILITY: If master dies, promote a replica
  2. READ SCALING: Route reads to replicas, writes to master
  3. DATA SAFETY: Multiple copies of data across servers

HOW REPLICATION WORKS:
  1. Replica connects to master: REPLICAOF master_host master_port
  2. Master performs background BGSAVE → sends RDB to replica
  3. Replica loads RDB → catches up with write log
  4. After initial sync: master streams new writes in real-time

CAVEAT: Replication is ASYNCHRONOUS by default
  → Short window where replica may have stale data
  → For cache/locks: acceptable (cache is already eventually consistent)
  → For critical data: use WAIT command to ensure N replicas received the write
```

3.5 Clustering

```
REDIS CLUSTER ARCHITECTURE:

  ┌──────────────────────────────────────────────────────────────┐
  │                       REDIS CLUSTER                          │
  │                                                              │
  │  ┌────────────┐   ┌────────────┐   ┌────────────┐           │
  │  │  Node A    │   │  Node B    │   │  Node C    │           │
  │  │ Slots 0-   │   │ Slots      │   │ Slots      │           │
  │  │ 5460       │   │ 5461-10922 │   │ 10923-16383│           │
  │  │            │   │            │   │            │           │
  │  │ Master     │   │ Master     │   │ Master     │           │
  │  │  ↓         │   │  ↓         │   │  ↓         │           │
  │  │ Replica A1 │   │ Replica B1 │   │ Replica C1 │           │
  │  └────────────┘   └────────────┘   └────────────┘           │
  └──────────────────────────────────────────────────────────────┘

HOW IT WORKS:
  1. Key space divided into 16384 hash slots
  2. Each key hashed: CRC16(key) mod 16384 → slot number
  3. Each master owns a range of slots
  4. Client connects to ANY node → redirected to correct node (MOVED)

EXAMPLE:
  SET product::1 "{...}"
  CRC16("product::1") mod 16384 = 8234
  Slot 8234 → Node B
  Client redirected to Node B → SET executed

THIS PROJECT: Single Redis node (development)
PRODUCTION: 3 masters + 3 replicas (minimum)
```

3.6 Eviction Policies

```
WHEN REDIS RUNS OUT OF MEMORY:

1. noeviction (default):
   → Reject writes with OOM error
   → Reads still work
   → USE: When data loss is unacceptable

2. allkeys-lru (RECOMMENDED for caching):
   → Evict Least Recently Used key from ALL keys
   → Best for cache use case
   → THIS PROJECT should use this

3. volatile-lru:
   → Evict LRU key only from keys WITH TTL set
   → Preserves keys without TTL (permanent data)

4. allkeys-lfu (Redis 4.0+):
   → Evict Least Frequently Used key from ALL keys
   → Better for workloads with hot/cold data

5. volatile-lfu:
   → LFU eviction only for keys with TTL

6. allkeys-random:
   → Randomly evict any key
   → Simple but suboptimal

7. volatile-random:
   → Random eviction only for keys with TTL

8. volatile-ttl:
   → Evict keys with shortest remaining TTL first

CONFIGURATION:
  maxmemory 256mb                    # Maximum memory Redis can use
  maxmemory-policy allkeys-lru        # Eviction policy
```

---

4. Microservices Integration Strategy

4.1 Redis in Product Service — Cache-Aside Pattern

```
FLOW: GET /api/products/{id}

  ┌──────────┐     ┌──────────────┐     ┌──────────┐
  │  Client  │────▶│   Product    │     │  Redis   │
  │          │     │   Service    │────▶│  Cache   │
  │          │     │              │     │          │
  │          │     │ @Cacheable   │     │          │
  │          │     └──────────────┘     └──────────┘
  │          │        │                     │
```

```
  │          │        │  CACHE HIT?          │
  │          │        │ ◄────────────────────┘
  │          │        │  YES → return cached ProductDto
  │          │        │  NO  → query MySQL ↓
  │          │        │
  │          │        ▼
  │          │     ┌──────────────┐
  │          │     │  MySQL       │
  │          │     │  product_db  │
  │          │     └──────────────┘
  │          │◄──────── return ProductDto (now cached for 10 min)

KEY DESIGN:
  Cache Name: "product"    → individual products
  Cache Name: "products"   → product list
  Key Pattern: product::1  → product with ID 1
  TTL: 10 min (individual), 5 min (lists)
  Serialization: Jackson JSON (human-readable in Redis CLI)
```

4.2 Redis in Inventory Service — Distributed Lock

```
FLOW: Reserve Stock for Order


  ┌──────────────┐     ┌──────────────┐     ┌──────────┐
  │  Inventory   │     │    Redis     │     │  MySQL   │
  │  Consumer    │────▶│   Lock       │     │ inv_db   │
  │ (Kafka)      │     │              │     │          │
  └──────────────┘     └──────────────┘     └──────────┘
          │                    │                  │
          │ 1. SETNX           │                  │
          │    lock:inv:42     │                  │
          │    UUID value      │                  │
          │    TTL=30s         │                  │
          │───────────────────▶│                  │
          │                    │                  │
          │ 2. Lock acquired?  │                  │
          │◄───────────────────│                  │
          │   YES              │                  │
          │                    │                  │
          │ 3. Read inventory  │                  │
          │────────────────────┼─────────────────▶│
          │   SELECT stock     │                  │
          │   WHERE product=42 │                  │
          │◄───────────────────┼──────────────────│
          │   stock = 50       │                  │
          │                    │                  │
          │ 4. Update stock    │                  │
          │────────────────────┼─────────────────▶│
          │   UPDATE stock=48  │                  │
          │   WHERE product=42 │                  │
          │                    │                  │
```

```
          | 5. DEL lock:inv:42 |                   |
          |──────────────────▶|                   |
          |  (only if UUID     |                   |
          |   matches)         |                   |
          |                    |                   |

SAFETY: Lock released ONLY by the holder (UUID check)
  → Prevents Thread B from accidentally releasing Thread A's lock
```

4.3 Redis in API Gateway — Rate Limiting

```
FLOW: Rate Limiting (100 requests/minute per user)

  Client → API Gateway → Check Redis Counter → Allow/Deny

  Resilience4j Rate Limiter Configuration:
    limit-for-period: 100      # 100 requests allowed
    limit-refresh-period: 1m   # per 1 minute window
    timeout-duration: 0s       # No waiting (immediate reject if exceeded)

  Under the hood (simplified):
    1. Client sends request with JWT → extract userId
    2. Redis: INCR ratelimit:userId:currentMinute
    3. If count <= 100 → ALLOW request
    4. If count > 100 → REJECT with HTTP 429 Too Many Requests
    5. Key expires after 1 minute (TTL = 60s)

  WHY REDIS FOR RATE LIMITING?
    - Centralized: Works across ALL gateway instances
    - Atomic: INCR is atomic → no race conditions
    - Fast: Sub-millisecond → negligible latency overhead
    - Auto-expire: TTL handles cleanup automatically
```

---

5. Step-by-Step Implementation Guide

Step 1 — Redis Setup with Docker

Docker Compose Configuration (from this project):

```yaml
redis:
  image: redis:7-alpine       # Redis 7.x (latest stable, Alpine for small image)
  container_name: order-system-redis
  ports:
    - "6379:6379"              # Default Redis port
  volumes:
    - redis-data:/data         # Persistent volume for RDB/AOF snapshots
  networks:
    - order-system-network     # Same network as all microservices
```

WHY redis:7-alpine?

- Alpine: 5MB base image vs 100MB+ Debian → faster pull, smaller footprint
- Redis 7.x: Latest stable with ACL improvements, multi-part AOF, function support

Redis CLI Basics:

```
# Connect to Redis
docker exec -it order-system-redis redis-cli

# Test connection
127.0.0.1:6379> PING
PONG

# Set a value
127.0.0.1:6379> SET greeting "Hello Redis"
OK

# Get a value
127.0.0.1:6379> GET greeting
"Hello Redis"

# View all keys
127.0.0.1:6379> KEYS *
1) "product::1"
2) "products::SimpleKey []"
3) "inventory:lock:42"

# Check TTL of a key
127.0.0.1:6379> TTL product::1
(integer) 542    # 542 seconds remaining

# View key type
127.0.0.1:6379> TYPE product::1
string

# View memory usage
127.0.0.1:6379> INFO memory
used_memory_human:1.24M
maxmemory_human:0B    # No limit set (unlimited)

# Monitor all commands in real-time
127.0.0.1:6379> MONITOR
OK
1614695423.456 [0 172.18.0.5:38012] "GET" "product::1"
1614695424.789 [0 172.18.0.6:39084] "SET" "inventory:lock:42" "uuid-abc" "NX" "EX" "30"
```

Production Redis Configuration (redis.conf):

```
# Memory management
maxmemory 256mb                    # Maximum memory limit
maxmemory-policy allkeys-lru       # Evict least recently used keys
```

```
# Persistence
save 300 100                        # RDB snapshot: every 5 min if 100+ changes
save 60 10000                       # RDB snapshot: every 1 min if 10000+ changes
appendonly yes                      # Enable AOF
appendfsync everysec                # Flush AOF to disk every second

# Security
requirepass ${REDIS_PASSWORD}       # Password authentication
# ACL: user product-service on >password ~product:* +get +set +del
# ACL: user inventory-service on >password ~inventory:* +get +set +del +setnx

# Network
bind 0.0.0.0                       # Listen on all interfaces (Docker)
timeout 300                         # Close idle connections after 5 min
tcp-keepalive 60                    # TCP keepalive every 60 seconds

# Slow log
slowlog-log-slower-than 10000      # Log commands slower than 10ms
slowlog-max-len 128                 # Keep last 128 slow commands
```

Step 2 — Spring Boot Caching Implementation

2.1 Maven Dependencies

```
<!-- Spring Data Redis -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<!-- Spring Cache Abstraction -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

2.2 Application Configuration (application.yml)

```yaml
spring:
  data:
    redis:
      host: localhost               # Redis server address
      port: 6379                    # Redis server port
  cache:
    type: redis                     # Use Redis as cache backend
    redis:
      time-to-live: 600000          # Default TTL: 10 minutes (milliseconds)
      cache-null-values: false      # Don't cache null results
```

WHY `cache-null-values: false` ?

- Prevents "cache penetration" — if a product ID doesn't exist, we don't want to cache `null`
- Without this: `GET /products/999` (doesn't exist) → caches null → subsequent requests hit cache but get null → confusing behavior
- With this: Every request for non-existent products goes to the database (which correctly throws "not found")

2.3 Redis Configuration Class

```java
@Configuration
public class RedisConfig {

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory connectionFactory) {
        // Default cache config with 10-minute TTL
        RedisCacheConfiguration defaultConfig = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))       // Keys expire after 10 min
                .disableCachingNullValues()              // Don't cache null
                .serializeKeysWith(                      // Keys as plain strings
                    SerializationPair.fromSerializer(new StringRedisSerializer()))
                .serializeValuesWith(                    // Values as JSON
                    SerializationPair.fromSerializer(new
GenericJackson2JsonRedisSerializer()));

        // Custom config for product list cache (shorter TTL)
        RedisCacheConfiguration listConfig = defaultConfig.entryTtl(Duration.ofMinutes(5));

        return RedisCacheManager.builder(connectionFactory)
                .cacheDefaults(defaultConfig)            // All caches: 10 min TTL
                .withCacheConfiguration("products", listConfig)  // "products": 5 min TTL
                .build();
    }
}
```

SERIALIZATION STRATEGY COMPARISON:

```
+-------------------------+-------------------------+-------------------------+
| Feature                 | JDK Serialization       | Jackson JSON (Used)     |
+-------------------------+-------------------------+-------------------------+
| Stored format           | Binary bytes            | Human-readable JSON     |
| Readability             | Unreadable in CLI       | Readable in Redis CLI   |
| Size                    | Larger (class metadata)| Smaller (no metadata)   |
| Cross-language          | Java only               | Any language            |
| Speed                   | Slightly faster         | Slightly slower         |
| Debugging               | Very difficult          | Easy (redis-cli GET)    |
| Version compatibility   | Breaks on class change | Resilient to changes    |
+-------------------------+-------------------------+-------------------------+

RECOMMENDATION: Always use Jackson JSON for application caching.
  → You can inspect cache contents with: redis-cli GET product::1
  → Output: {"id":1,"name":"Laptop","price":999.99,"category":"Electronics"}
```

WHY per-cache TTL?

```
Product by ID (10 min TTL):
  - Products change infrequently (admin updates)
  - Longer TTL = more cache hits = fewer DB queries
  - Invalidated on update/delete via @CacheEvict/@CachePut

Product List (5 min TTL):
  - List changes when ANY product is created, updated, or deleted
  - Shorter TTL = faster reflection of new products
  - Also explicitly evicted on create/update/delete
```

2.4 Using @Cacheable, @CachePut, @CacheEvict

```java
@Service
public class ProductService {

    // ────────────────────────────────────────────────────────────
    // READ: Check cache first → DB only on cache miss
    // ────────────────────────────────────────────────────────────

    @Cacheable(value = "product", key = "#id")
    public ProductDto getProductById(Long id) {
        log.info("CACHE MISS — Fetching product from DB with ID: {}", id);
        // This log ONLY appears when Redis doesn't have the key
        // On cache hit, this method is NEVER CALLED (Spring returns cached value)
        Product product = repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Product not found"));
        return mapEntityToDto(product);
    }

    @Cacheable(value = "products")
    public List<ProductDto> getAllProducts() {
        log.info("CACHE MISS — Fetching all products from DB");
        return repository.findByActiveTrue().stream()
                .map(this::mapEntityToDto).collect(Collectors.toList());
    }

    // ────────────────────────────────────────────────────────────
    // CREATE: Evict product list cache (new product added)
    // ────────────────────────────────────────────────────────────

    @CacheEvict(value = "products", allEntries = true)
    public ProductDto createProduct(ProductDto dto) {
        // After this method completes, "products" cache is cleared
        // Next getAllProducts() call → cache miss → fresh DB query
        Product saved = repository.save(product);
        return mapEntityToDto(saved);
    }

    // ────────────────────────────────────────────────────────────
    // UPDATE: Update individual cache + evict list cache
    // ────────────────────────────────────────────────────────────
```

```java
    @CachePut(value = "product", key = "#id")
    @CacheEvict(value = "products", allEntries = true)
    public ProductDto updateProduct(Long id, ProductDto dto) {
        // @CachePut: updates "product::id" in Redis with the return value
        // @CacheEvict: clears "products" list (now stale)
        Product product = repository.findById(id).orElseThrow();
        mapDtoToEntity(dto, product);
        return mapEntityToDto(repository.save(product));
    }

    // ────────────────────────────────────────────────────────────
    // DELETE: Evict both individual and list caches
    // ────────────────────────────────────────────────────────────

    @CacheEvict(value = { "product", "products" }, allEntries = true)
    public void deleteProduct(Long id) {
        // Clears BOTH "product" and "products" caches entirely
        Product product = repository.findById(id).orElseThrow();
        product.setActive(false);            // Soft delete
        repository.save(product);
    }
}
```

ANNOTATION BEHAVIOR MAP:

```
+──────────────+──────────────────────────────────────────────────────────+
| Annotation   | Behavior                                                 |
+──────────────+──────────────────────────────────────────────────────────+
| @Cacheable   | BEFORE method: check cache                               |
|              |   → HIT: return cached value, method NOT called          |
|              |   → MISS: call method, store result in cache, return     |
+──────────────+──────────────────────────────────────────────────────────+
| @CachePut    | ALWAYS calls the method, ALWAYS updates the cache        |
|              | Use for: updates (want to execute AND update cache)       |
+──────────────+──────────────────────────────────────────────────────────+
| @CacheEvict  | AFTER method: remove entry/entries from cache            |
|              | allEntries=true: clear ALL entries in the cache           |
|              | key="#id": remove only the specific entry                 |
+──────────────+──────────────────────────────────────────────────────────+
```

Step 3 — Distributed Lock Implementation

3.1 Why Distributed Locking is Required

```
SCENARIO: 100 concurrent requests to reserve the last item

WITHOUT DISTRIBUTED LOCK (database only):
  └── Instance A reads: stock = 1
  └── Instance B reads: stock = 1    (same time, different server)
  └── Instance A: stock >= 1 → reserve → stock = 0 ✓
  └── Instance B: stock >= 1 → reserve → stock = -1 ❌ OVERSOLD!
```

```
WITH DISTRIBUTED LOCK (Redis SETNX):
  └── Instance A: SETNX lock:inventory:42 → ACQUIRED
  └── Instance B: SETNX lock:inventory:42 → BLOCKED (key exists)
  └── Instance A: read stock=1 → reserve → stock=0 → release lock
  └── Instance B: SETNX lock:inventory:42 → ACQUIRED (key deleted)
  └── Instance B: read stock=0 → INSUFFICIENT → reject ✓

WHY NOT DATABASE-LEVEL LOCK?
  1. SELECT FOR UPDATE: Locks the row in MySQL → works BUT:
     - Holds DB connection during entire operation
     - Multiple operations = multiple row locks = potential deadlock
     - Doesn't work across different databases (microservices)
  2. Redis lock: faster (~0.1ms), doesn't consume DB connections,
     works across ALL instances of ALL services
```

3.2 SETNX Implementation

```java
// Redis Lock Configuration
@Configuration
public class RedisLockConfig {
    @Bean
    public StringRedisTemplate stringRedisTemplate(RedisConnectionFactory factory) {
        return new StringRedisTemplate(factory);
    }
}

// Lock Usage in InventoryService
public boolean reserveStock(Long productId, int quantity) {
    String lockKey = "inventory:lock:" + productId;   // Per-product lock
    String lockValue = UUID.randomUUID().toString();   // Unique owner ID

    try {
        // ── STEP 1: Acquire Lock ──────────────────────────
        // SET lockKey lockValue NX EX 30
        //   NX = Set if Not eXists → atomic lock acquisition
        //   EX 30 = auto-expire in 30 seconds → prevent deadlock
        Boolean acquired = redisTemplate.opsForValue()
                .setIfAbsent(lockKey, lockValue, Duration.ofSeconds(30));

        if (Boolean.FALSE.equals(acquired)) {
            log.warn("Could not acquire lock for productId={}", productId);
            return false;  // Another transaction in progress
        }

        // ── STEP 2: Critical Section ──────────────────────
        // Only ONE thread/instance reaches here at a time
        return doReserveStock(productId, quantity);

    } finally {
        // ── STEP 3: Release Lock (SAFE) ───────────────────
```

```
        // ONLY delete if WE still hold the lock (UUID check)
        String currentValue = redisTemplate.opsForValue().get(lockKey);
        if (lockValue.equals(currentValue)) {
            redisTemplate.delete(lockKey);
        }
    }
}
```

CRITICAL: WHY UUID LOCK VALUE?

```
SCENARIO WITHOUT UUID CHECK:

  Thread A: SETNX lock:inv:42 "anything" → acquired, TTL=30s
  Thread A: starts processing (takes longer than expected)
  ... 30 seconds pass ... lock auto-expires
  Thread B: SETNX lock:inv:42 "anything" → acquired (A's lock expired)
  Thread A: finishes processing → DEL lock:inv:42
  Thread A just DELETED Thread B's lock! ❌
  Thread C: SETNX lock:inv:42 "anything" → acquired
  Now BOTH Thread B and Thread C think they have the lock!

SCENARIO WITH UUID CHECK:

  Thread A: SETNX lock:inv:42 "uuid-A" → acquired, TTL=30s
  Thread A: starts processing
  ... 30 seconds pass ... lock auto-expires
  Thread B: SETNX lock:inv:42 "uuid-B" → acquired (A's lock expired)
  Thread A: finishes → GET lock:inv:42 → "uuid-B" ≠ "uuid-A" → DON'T DELETE
  Thread B still holds the lock safely ✓
```

3.3 Two-Layer Concurrency Protection

```
THIS PROJECT'S DEFENSE-IN-DEPTH APPROACH:

Layer 1: Redis Distributed Lock (SETNX)
  → First line of defense
  → Prevents concurrent access across ALL service instances
  → If Redis is down → lock acquisition returns false → request rejected safely

Layer 2: JPA Optimistic Locking (@Version)
  → Second safety net
  → Inventory entity has @Version field
  → On save(): Hibernate checks if version changed since read
  → If changed → OptimisticLockingFailureException → retry (up to 3 times)
  → Catches edge cases where Redis lock fails (network partition, TTL race)

WHY BOTH?
  Redis lock alone: What if Redis has a network glitch?
    → Lock fails to acquire/release → fall through to optimistic lock
  Optimistic lock alone: What if 100 threads all read stock=1?
    → Only 1 succeeds on write, 99 retry (expensive!)
```

```
    → Redis lock prevents 99 threads from even reaching the DB

TOGETHER: 99% handled by Redis lock (fast), 1% caught by optimistic lock (safe)
```

```
// Retry logic for optimistic locking failures
for (int attempt = 1; attempt <= MAX_RETRY_ATTEMPTS; attempt++) {
    try {
        return doReserveStock(productId, quantity);
    } catch (OptimisticLockingFailureException e) {
        log.warn("Optimistic lock conflict, attempt {}/{}", attempt, MAX_RETRY_ATTEMPTS);
        if (attempt == MAX_RETRY_ATTEMPTS) return false;
        Thread.sleep(100L * (1L << (attempt - 1)));  // Exponential backoff: 100ms, 200ms,
400ms
    }
}
```

3.4 RedLock Algorithm (Production Enhancement)

```
PROBLEM: Single Redis instance is a single point of failure.
  If Redis dies → all locks are lost → multiple threads enter critical section.

SOLUTION: RedLock (Distributed Lock across N Redis instances)

  Client acquires lock on MAJORITY (N/2+1) of N independent Redis instances.
  If majority acquired → lock is valid.
  If not → release all acquired locks → retry.

EXAMPLE WITH 5 REDIS INSTANCES:
  1. Client tries SETNX on Redis A → acquired
  2. Client tries SETNX on Redis B → acquired
  3. Client tries SETNX on Redis C → acquired  (3/5 = majority ✓)
  4. Client tries SETNX on Redis D → failed
  5. Client tries SETNX on Redis E → failed

  Result: Lock acquired (3 out of 5 = majority)
  Even if Redis D and E die → client still holds 3 locks ✓

IMPLEMENTATION (Redisson library):
  RLock lock1 = redissonClient1.getLock("inventory:lock:42");
  RLock lock2 = redissonClient2.getLock("inventory:lock:42");
  RLock lock3 = redissonClient3.getLock("inventory:lock:42");

  RedissonRedLock multiLock = new RedissonRedLock(lock1, lock2, lock3);
  multiLock.lock();
  try {
      // Critical section
  } finally {
      multiLock.unlock();
  }
```

Step 4 — Rate Limiting Implementation

4.1 This Project's Rate Limiter

```yaml
# API Gateway — Resilience4j Rate Limiter (backed by Redis)
resilience4j:
  ratelimiter:
    configs:
      default:
        limit-for-period: 100         # 100 requests per window
        limit-refresh-period: 1m       # Window = 1 minute
        timeout-duration: 0s           # No waiting; immediate reject
```

4.2 Token Bucket Algorithm (Conceptual)

```
TOKEN BUCKET ALGORITHM:

  ┌──────────────────────────────┐
  │      TOKEN BUCKET            │
  │                              │
  │  Tokens: ████████▒           │  (8 of 10 tokens available)
  │                              │
  │  Refill rate: 10 tokens/minute │  (adds tokens over time)
  │  Bucket size: 10 tokens      │  (maximum tokens)
  └──────────────────────────────┘

  REQUEST ARRIVES:
    If tokens > 0 → consume 1 token → ALLOW
    If tokens = 0 → DENY (429 Too Many Requests)

  Redis Implementation:
    Key: ratelimit:{userId}:{window}
    Value: remaining token count
    TTL: window duration (1 minute)

    DECR ratelimit:user7:202603031200
    → If result >= 0 → ALLOW
    → If result < 0 → DENY + reset to 0
```

4.3 Sliding Window Algorithm (Alternative)

```
SLIDING WINDOW (More accurate for burst prevention):

  Fixed Window Problem:
    Window 1: [00:00 - 01:00] → 100 requests (limit=100) ✓
    Window 2: [01:00 - 02:00] → 100 requests (limit=100) ✓
    BUT: 100 requests at 00:59 + 100 requests at 01:01 = 200 in 2 seconds ❌

  Sliding Window Solution:
    At time 01:30:
```

```
    Current window weight: 30/60 * 100 = 50
    Previous window weight: 30/60 * 85 = 42.5
    Total: 92.5 < 100 → ALLOW

 Redis Implementation (Sorted Set):
   ZADD ratelimit:{userId} {timestamp} {requestId}
   ZREMRANGEBYSCORE ratelimit:{userId} 0 {timestamp - window}
   ZCARD ratelimit:{userId}
   → If count < limit → ALLOW
   → If count >= limit → DENY
```

4.4 Lua Script for Atomic Rate Limiting (Production)

```lua
-- Atomic rate limiting (executed as single Redis operation)
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2])

local current = redis.call('INCR', key)
if current == 1 then
    redis.call('EXPIRE', key, window)     -- Set TTL on first request
end

if current > limit then
    return 0  -- DENY
else
    return 1  -- ALLOW
end
```

```java
// Spring Boot Lua script execution
@Component
public class RedisRateLimiter {
    private final StringRedisTemplate redisTemplate;
    private final RedisScript<Long> rateLimitScript;

    public boolean isAllowed(String userId, int limit, int windowSeconds) {
        String key = "ratelimit:" + userId + ":" + (System.currentTimeMillis() /
(windowSeconds * 1000));
        Long result = redisTemplate.execute(rateLimitScript,
                List.of(key), String.valueOf(limit), String.valueOf(windowSeconds));
        return result != null && result == 1;
    }
}
```

Step 5 — Redis Pub/Sub & Streams

5.1 Redis Pub/Sub Architecture

```
PUBLISHER → CHANNEL → SUBSCRIBER(S)
```

```
  ┌──────────────┐          ┌──────────────┐
  │ Order Service │─publish──│   Redis      │──subscribe──▶ Notification Service
  │               │          │  Channel:    │──subscribe──▶ Analytics Service
  │               │          │ "order-events"│──subscribe──▶ Dashboard Service
  └──────────────┘          └──────────────┘

DIFFERENCE FROM KAFKA:
  Pub/Sub: Fire-and-forget. If subscriber is offline, message is LOST.
  Kafka: Persistent. Messages retained for days/weeks. Consumer replays.

USE CASE IN THIS PROJECT:
  Real-time dashboard updates (online users can see live order count updates)
  NOT suitable for: order processing (need guaranteed delivery → use Kafka)
```

```java
// Publisher
redisTemplate.convertAndSend("order-events",
    "{\"orderId\": 42, \"status\": \"CREATED\"}");

// Subscriber Configuration
@Configuration
public class RedisPubSubConfig {
    @Bean
    public RedisMessageListenerContainer container(RedisConnectionFactory factory,
            MessageListenerAdapter adapter) {
        RedisMessageListenerContainer container = new RedisMessageListenerContainer();
        container.setConnectionFactory(factory);
        container.addMessageListener(adapter, new ChannelTopic("order-events"));
        return container;
    }
}
```

5.2 Redis Streams (Persistent Messaging)

```
REDIS STREAMS vs PUB/SUB:
  Pub/Sub: No persistence, no consumer groups, fire-and-forget
  Streams: Persistent log, consumer groups, acknowledgment, replay

STREAM STRUCTURE:
  Stream: "order-events"
    Entry 1: {1614695423456-0} → {orderId: 42, status: CREATED}
    Entry 2: {1614695424789-0} → {orderId: 43, status: CREATED}
    Entry 3: {1614695425123-0} → {orderId: 44, status: CANCELLED}

  Consumer Group: "notification-group"
    Consumer A: reads Entry 1, ACK
    Consumer B: reads Entry 2, ACK
    Consumer C: reads Entry 3, ACK (pending if not ACK'd)

WHEN TO USE:
  ✓ Lightweight event streaming (alternative to Kafka for low-volume use cases)
  ✓ Task queues with acknowledgment
```

```
  ✓ Activity feeds

WHEN NOT TO USE:
  ✗ High-volume event processing (Kafka is better)
  ✗ Cross-datacenter replication (Kafka has MirrorMaker)
  ✗ Schema enforcement (Kafka has Schema Registry)
```

---

6. Code Walkthrough

6.1 Complete Product Caching Flow

```
USER: GET /api/products/1 (First Request — CACHE MISS)
═══════════════════════════════════════════════════

  1. Request hits ProductController.getProductById(1)
  2. Spring AOP intercepts @Cacheable(value="product", key="#id")
  3. Spring checks Redis: GET product::1
  4. Redis returns: (nil)  → CACHE MISS
  5. Spring calls ProductService.getProductById(1)
     → Log: "CACHE MISS — Fetching product from DB with ID: 1"
     → MySQL: SELECT * FROM products WHERE id = 1
     → Returns ProductDto{id=1, name="Laptop", price=999.99}
  6. Spring stores in Redis:
     SET product::1 '{"@class":"...ProductDto","id":1,"name":"Laptop","price":999.99}'
     TTL: 600 seconds (10 minutes)
  7. Returns ProductDto to client

USER: GET /api/products/1 (Second Request — CACHE HIT)
═══════════════════════════════════════════════════

  1. Request hits ProductController.getProductById(1)
  2. Spring AOP intercepts @Cacheable
  3. Spring checks Redis: GET product::1
  4. Redis returns: '{"@class":"...ProductDto","id":1,"name":"Laptop","price":999.99}'
  5. → CACHE HIT!
  6. Spring deserializes JSON → ProductDto
  7. ProductService.getProductById() IS NEVER CALLED
     → No log "CACHE MISS" appears
     → No MySQL query executed
  8. Returns ProductDto to client

  PERFORMANCE:
    Cache miss: ~5ms (Redis) + ~5ms (MySQL) = ~10ms
    Cache hit:  ~0.5ms (Redis only)
    Improvement: 20x faster on subsequent requests

ADMIN: PUT /api/admin/products/1 (Update — CACHE INVALIDATION)
═══════════════════════════════════════════════════

  1. Request hits ProductController.updateProduct(1, dto)
  2. Spring calls ProductService.updateProduct(1, dto)
```

```
  3. MySQL: UPDATE products SET name='Laptop Pro' WHERE id=1
  4. Returns updated ProductDto{id=1, name="Laptop Pro"}
  5. @CachePut: Redis SET product::1 (updated JSON)
  6. @CacheEvict: Redis DEL products::* (list cache cleared)

  After update:
    GET /products/1 → returns "Laptop Pro" (from updated cache)
    GET /products   → cache miss → fresh DB query (list rebuilds)
```

6.2 Complete Distributed Lock Flow

```
KAFKA: order-created event arrives for orderId=42, productId=7, qty=2
═══════════════════════════════════════════════════════════════════

  1. InventoryConsumer.handleOrderCreated(event)
  2. Calls inventoryService.reserveStock(productId=7, quantity=2)

  ── Redis Lock Phase ──────────────────────────────────────
  3. Generate lockKey = "inventory:lock:7"
  4. Generate lockValue = "uuid-abc-123" (unique per call)
  5. Redis: SET inventory:lock:7 "uuid-abc-123" NX EX 30
     → NX: Only set if key doesn't exist (atomic)
     → EX 30: Auto-expire in 30 seconds
  6. Redis returns: true → LOCK ACQUIRED

  ── Critical Section ──────────────────────────────────────
  7. MySQL: SELECT * FROM inventory WHERE product_id=7
     → Returns: Inventory{productId=7, quantity=50, reserved=10, version=3}
  8. availableQuantity = 50 - 10 = 40
  9. 40 >= 2 → SUFFICIENT
  10. inventory.setReservedQuantity(10 + 2 = 12)
  11. MySQL: UPDATE inventory SET reserved_quantity=12, version=4
           WHERE product_id=7 AND version=3
     → @Version check: version=3 matches → SUCCESS
     → If version ≠ 3 → OptimisticLockingFailureException → retry

  ── Lock Release ──────────────────────────────────────────
  12. Redis: GET inventory:lock:7 → "uuid-abc-123"
  13. "uuid-abc-123" == "uuid-abc-123" → MATCH
  14. Redis: DEL inventory:lock:7
  15. Lock released safely

  16. Return true → InventoryConsumer publishes inventory-updated (RESERVED)

CONCURRENT REQUEST (productId=7) DURING STEP 7-11:
  Thread B: SET inventory:lock:7 "uuid-def-456" NX EX 30
  Redis returns: false (key exists — Thread A holds lock)
  Thread B: returns false → stock NOT reserved → consumer publishes INSUFFICIENT_STOCK
  → No race condition, no overselling ✓
```

---

7. Caching Design Patterns

7.1 Cache-Aside (Lazy Loading) — IMPLEMENTED

```
READ:
  1. Application checks cache first
  2. If FOUND (hit) → return cached data
  3. If NOT FOUND (miss) → query database → store in cache → return

WRITE:
  1. Write to database
  2. Invalidate (evict) cache
  3. Next read → cache miss → fresh data loaded from DB

ADVANTAGES:
  ✓ Only frequently accessed data is cached (lazy)
  ✓ Cache failure doesn't prevent reads (fallback to DB)
  ✓ Easy to implement with Spring @Cacheable/@CacheEvict

DISADVANTAGES:
  ✗ First request after eviction is always slow (cache miss)
  ✗ Data can be stale until TTL expires or manual eviction

THIS PROJECT: Uses Cache-Aside for all product data.
```

7.2 Write-Through

```
WRITE:
  1. Application writes to cache AND database synchronously
  2. Cache always has the latest data

ADVANTAGES:
  ✓ Cache never stale (always in sync with DB)
  ✓ Reads always hit cache (no cold misses after write)

DISADVANTAGES:
  ✗ Write latency doubled (cache + DB on every write)
  ✗ Caching data that may never be read

WHEN TO USE:
  Read-heavy workloads where data MUST be current (e.g., user profile display)
```

7.3 Write-Behind (Write-Back)

```
WRITE:
  1. Application writes to cache immediately (fast response)
  2. Background process writes to database asynchronously (eventually)

ADVANTAGES:
  ✓ Ultra-fast writes (only cache, no DB wait)
```

```
  ✓ Batches DB writes for efficiency

DISADVANTAGES:
  ✗ Data loss risk if cache fails before DB write
  ✗ Complex implementation (background sync)

WHEN TO USE:
  Write-heavy workloads with acceptable eventual consistency
  (e.g., analytics event tracking, view counters)
```

7.4 Read-Through

```
READ:
  1. Application requests data from cache
  2. If MISS → cache itself queries the database
  3. Cache stores and returns the result

DIFFERENCE FROM CACHE-ASIDE:
  Cache-Aside: Application manages cache population
  Read-Through: Cache manages cache population (cache as data layer)

WHEN TO USE:
  When cache is the primary data access layer (CDN, Cloudflare cache)
```

7.5 Refresh-Ahead

```
READ:
  1. Cache proactively refreshes keys BEFORE they expire
  2. Based on key access pattern: if key accessed frequently, refresh early

ADVANTAGES:
  ✓ Eliminates cold miss penalty (data always fresh in cache)
  ✓ Smooth performance (no sudden latency spikes)

DISADVANTAGES:
  ✗ Wastes resources refreshing keys that may not be needed
  ✗ Complex to implement (predict which keys to refresh)

WHEN TO USE:
  Critical data with strict latency requirements
  (e.g., stock prices, trending products)
```

---

8. Distributed Systems Challenges & Solutions

8.1 Cache Stampede (Thundering Herd)

```
PROBLEM:
  Popular key expires → 1000 concurrent requests all experience cache miss
  → 1000 simultaneous database queries for the same data
```

```
  → Database overwhelmed → cascading failure

  Timeline:
    product::1 expires at 12:10:00
    12:10:00.001: Thread A → cache miss → query DB
    12:10:00.002: Thread B → cache miss → query DB
    12:10:00.003: Thread C → cache miss → query DB
    ...
    12:10:00.100: 1000 threads → 1000 DB queries for the SAME product!

HOW TO DETECT:
  - Sudden spike in DB connections after cache expiry
  - Response time spikes at predictable intervals (TTL boundaries)
  - Database CPU/connections maxed out periodically

SOLUTIONS:

  1. MUTEX LOCK (best for this project):
     Only ONE thread queries DB; others wait for cache to be populated.
     @Cacheable + @CacheConfig(sync = true)

  2. STAGGERED TTL (simple):
     TTL = baseTime + random(0, 120 seconds)
     Keys expire at different times → stampede avoided

  3. WARM-UP ON STARTUP:
     Preload popular keys on service startup.
     @PostConstruct → query DB → populate cache

  4. BACKGROUND REFRESH:
     Refresh keys in background BEFORE expiry.
     Scheduler checks keys nearing TTL → refreshes proactively.
```

8.2 Cache Penetration

```
PROBLEM:
  Requests for data that DOESN'T EXIST in DB.
  Every request → cache miss → DB query → empty result → nothing cached
  → DB queried on EVERY request for non-existent data

  Example: GET /products/99999999 (doesn't exist)
    Cache miss → DB: SELECT WHERE id=99999999 → empty
    No result to cache → next request → same cycle
    Attacker sends 10000 req/s for random non-existent IDs → DB overloaded

HOW TO DETECT:
  - High cache miss rate for specific key patterns
  - DB queries returning empty results repeatedly
  - Unusual request patterns (sequential or random IDs)

SOLUTIONS:
```

```
  1. CACHE NULL VALUES (with short TTL):
     If DB returns empty → cache null for 60 seconds
     config: spring.cache.redis.cache-null-values=true (we disable this to avoid confusion)

  2. BLOOM FILTER (best for high-traffic systems):
     In-memory probabilistic data structure
     Check: "Does this key POSSIBLY exist?"
     If NO → reject immediately, never hit DB
     If YES → query DB (small false positive rate)

  3. INPUT VALIDATION:
     Reject obviously invalid IDs before reaching cache/DB
     if (id <= 0 || id > MAX_PRODUCT_ID) throw new BadRequest();

  4. RATE LIMITING (this project):
     API Gateway rate limiter: 100 req/min → prevents abuse
```

8.3 Cache Avalanche

```
PROBLEM:
  Large number of keys expire at the SAME TIME → massive DB load

  Scenario: Service starts, loads 10000 products with TTL=10min
  → After exactly 10 minutes, ALL 10000 keys expire simultaneously
  → 10000 concurrent DB queries → database crashes

HOW TO DETECT:
  - Periodic performance degradation (every TTL interval)
  - All cache keys have the same expiry timestamp
  - DB load spikes at predictable intervals

SOLUTIONS:

  1. STAGGERED TTL (best):
     TTL = baseTTL + random(0, baseTTL * 0.2)
     10 min + random(0, 2 min) → keys expire between 10-12 minutes

  2. MULTI-TIER CACHING:
     L1: In-process cache (Caffeine, 1 min TTL)
     L2: Redis (10 min TTL)
     Even if Redis keys expire, L1 absorbs some load

  3. CIRCUIT BREAKER ON DB:
     If DB load exceeds threshold → return degraded response
     Prevents DB crash, allows gradual recovery

  4. WARM CACHE BEFORE RESTART:
     Before rolling restart, populate cache with hot keys
     docker exec redis-cli --pipe < hot_keys_dump.txt
```

8.4 Data Inconsistency

```
PROBLEM:
  Cache has stale data that doesn't match the database.

  Timeline:
    12:00:00 — Product price = $999 (DB and cache)
    12:05:00 — Admin updates price to $899 in DB
    12:05:01 — Cache still has $999 (not invalidated)
    12:05:02 — User sees $999 from cache → buys at wrong price!

HOW TO DETECT:
  - Customer complaints about wrong data
  - Comparing cache vs DB values periodically
  - Monitoring @CacheEvict failure rates

SOLUTIONS (This Project):

  1. @CacheEvict ON WRITE (implemented):
     Every updateProduct() → @CacheEvict("products") clears stale cache
     Every deleteProduct() → @CacheEvict("product", "products") clears both

  2. @CachePut ON UPDATE (implemented):
     updateProduct() → @CachePut("product", key="#id") updates the specific key
     Cache IMMEDIATELY reflects the new value

  3. TTL AS SAFETY NET (implemented):
     Even if eviction fails, cache expires in 10 minutes
     Maximum staleness: 10 minutes (acceptable for product catalog)

  4. LISTEN TO DATABASE BINLOG (advanced):
     Use Debezium to capture DB changes → invalidate Redis keys automatically
     Zero application-level cache management needed
```

8.5 Memory Overflow

```
PROBLEM:
  Redis runs out of memory → OOM → crashes or rejects writes

HOW TO DETECT:
  - Redis INFO memory: used_memory > maxmemory
  - Redis SLOWLOG shows eviction-related latency
  - Application errors: OOM command not allowed when used memory > 'maxmemory'

SOLUTIONS:

  1. SET MAXMEMORY:
     maxmemory 256mb
     maxmemory-policy allkeys-lru

  2. SET TTL ON ALL KEYS:
```

```
     Every cached value should have a TTL
     Never store permanent data in Redis without TTL (unless intended)

  3. MONITOR KEY SIZE:
     redis-cli --bigkeys → find largest keys
     redis-cli MEMORY USAGE product::1 → check individual key size

  4. AVOID LARGE VALUES:
     Don't cache entire product lists if they're 10MB
     Cache individual products + paginated results
```

8.6 Hot Keys Problem

```
PROBLEM:
  A single key receives disproportionally high traffic → single Redis instance bottleneck

  Example: "products" cache (getAllProducts) hit by every user
  → All requests go to the same Redis key → single thread saturated

HOW TO DETECT:
  - Redis SLOWLOG shows same key repeatedly
  - redis-cli --hotkeys (Redis 4.0+) shows frequency
  - Uneven load across Redis cluster nodes

SOLUTIONS:

  1. LOCAL CACHE (L1):
     Caffeine in-process cache (1 min TTL) → handle 90% of reads locally
     Redis only queried on L1 miss

  2. KEY REPLICATION:
     Spread hot key across multiple keys: product_shard_1, product_shard_2
     Load balancer selects random shard

  3. READ REPLICAS:
     Route reads to Redis replicas → distribute load
     Only writes go to master
```

8.7 Network Latency

```
PROBLEM:
  Redis is remote → network round-trip adds latency
  Typical: 0.1ms (local) vs 1-5ms (cross-AZ) vs 50ms+ (cross-region)

SOLUTIONS:

  1. CO-LOCATE Redis with application servers (same AZ)
  2. USE PIPELINE for batch operations:
     redisTemplate.executePipelined() → send multiple commands in one round-trip
  3. USE CONNECTION POOL:
     spring.data.redis.lettuce.pool.max-active: 16
```

```
    spring.data.redis.lettuce.pool.min-idle: 4
  4. LOCAL CACHE for extremely hot data (Caffeine L1 + Redis L2)
```

8.8 Redis Failover Issues

```
PROBLEM:
  Redis master dies → data loss (async replication) → stale locks released

SOLUTIONS:
  1. Redis Sentinel: Automatic failover in < 30 seconds
  2. Redis Cluster: Automatic shard redistribution
  3. Application resilience: Degrade gracefully when Redis is down
     → @Cacheable: fall back to DB (slower but operational)
     → Distributed lock: reject request safely (return false)
```

8.9 Split Brain Problem

```
PROBLEM:
  Network partition splits Redis Sentinel into two groups
  Both groups think the other is down → both elect a master
  → Two masters → writes to both → DATA DIVERGENCE

SOLUTIONS:
  1. min-replicas-to-write 1: Master requires at least 1 replica to accept writes
  2. min-replicas-max-lag 10: Replica must be within 10 seconds of master
  3. Odd number of Sentinels (3 or 5) for proper majority voting
```

8.10 Large Object Caching Risks

```
PROBLEM:
  Caching a 10MB object → consumes huge amount of memory
  30 such objects → 300MB → most of maxmemory used by few keys

HOW TO DETECT:
  redis-cli --bigkeys → identifies largest keys
  redis-cli DEBUG OBJECT <key> → shows serialized size

SOLUTIONS:
  1. COMPRESS before caching: Snappy/LZ4 compression on JSON
  2. PAGINATE: Don't cache full lists; cache pages of 50 items
  3. SELECTIVE CACHING: Only cache hot objects (LFU eviction)
  4. REFERENCE PATTERN: Cache references (IDs), not full objects
```

---

9. Performance Tuning Guide

9.1 Connection Pool Optimization

```yaml
# Spring Boot Redis connection pool (Lettuce)
spring:
```

```yaml
  data:
    redis:
      host: localhost
      port: 6379
      timeout: 2000ms                    # Command timeout
      connect-timeout: 1000ms            # Connection timeout
      lettuce:
        pool:
          max-active: 16                 # Maximum active connections
          max-idle: 8                    # Maximum idle connections
          min-idle: 4                    # Minimum idle connections (warm pool)
          max-wait: 2000ms               # Wait time before connection timeout
```

CONNECTION POOL SIZING:

```
FORMULA: max-active = (peak_concurrent_requests / request_duration_ms) * 2

Example:
  Peak: 500 concurrent requests
  Redis command: ~1ms average
  max-active = (500 / 1) * 2 = 1000 connections (way too many)

  REALITY: Most frameworks multiplex connections efficiently
  RECOMMENDATION: 8-32 connections for typical microservices

SIGNS OF UNDER-PROVISIONING:
  - Redis timeouts in logs
  - Thread blocked waiting for connection
  - max-wait exceeded errors

SIGNS OF OVER-PROVISIONING:
  - Most connections sit idle
  - Redis shows many connected clients (INFO clients)
  - Wasted memory and file descriptors
```

9.2 Pipelining (Batch Operations)

```java
// WITHOUT pipeline: 100 separate round-trips to Redis
for (int i = 0; i < 100; i++) {
    redisTemplate.opsForValue().get("key:" + i);
}
// Time: 100 × 0.5ms = 50ms

// WITH pipeline: 1 round-trip for all 100 commands
List<Object> results = redisTemplate.executePipelined((RedisCallback<Object>) connection ->
{
    for (int i = 0; i < 100; i++) {
        connection.stringCommands().get(("key:" + i).getBytes());
    }
    return null;
```

```
});
// Time: 1 × 0.5ms + 0.001ms × 100 = ~0.6ms (83x faster!)
```

WHEN TO USE PIPELINE:

- Batch reads (warm cache on startup)
- Batch writes (bulk cache invalidation)
- Counters (increment multiple keys)
- NOT suitable when commands depend on previous results

9.3 Compression Strategy

```
// Large JSON values → compress before caching

@Bean
public RedisCacheConfiguration cacheConfiguration() {
    return RedisCacheConfiguration.defaultCacheConfig()
            .serializeValuesWith(
                SerializationPair.fromSerializer(
                    new GenericJackson2JsonRedisSerializer(
                        new ObjectMapper().configure(
                            SerializationFeature.INDENT_OUTPUT, false))))  // Compact JSON
            .entryTtl(Duration.ofMinutes(10));
}

// For very large values (> 1KB):
// Consider application-level compression
public byte[] compress(String json) {
    return Snappy.compress(json.getBytes(StandardCharsets.UTF_8));
}

public String decompress(byte[] compressed) {
    return new String(Snappy.uncompress(compressed), StandardCharsets.UTF_8);
}

// Compression savings (typical JSON):
//   Uncompressed: 2048 bytes
//   Snappy:       820 bytes (60% reduction)
//   GZIP:         410 bytes (80% reduction, but 5x slower)
```

9.4 Key Design Best Practices

```
KEY NAMING CONVENTIONS:
  ┌──────────────────────────────────────────────────────────────┐
  │ Pattern: {service}:{entity}:{identifier}                     │
  │                                                               │
  │ Examples:                                                     │
  │   product-service cache:  product::1, products::SimpleKey []  │
  │   inventory lock:         inventory:lock:42                   │
  │   rate limiter:           ratelimit:user7:202603031200        │
  │   session:                session:jwt:userId:7                │
```

```
  │   idempotency check:      processed:event:uuid-abc-123          │
  └─────────────────────────────────────────────────────────────────┘


BEST PRACTICES:
  ✓ Use colons (:) as separators (Redis convention)
  ✓ Use short but descriptive names
  ✓ Include entity type AND identifier
  ✓ Set TTL on ALL cache keys (prevent memory leaks)
  ✗ Don't use very long keys (increases memory usage)
  ✗ Don't use special characters (spaces, newlines)
  ✗ Don't store large values without compression
```

9.5 Memory Optimization

```
STRATEGIES TO REDUCE MEMORY USAGE:

1. USE HASHES FOR SMALL OBJECTS:
   Instead of: SET user:1:name "Alice", SET user:1:email "alice@..."
   Use:        HSET user:1 name "Alice" email "alice@..."
   → 10x less memory (Redis hash encoding optimization)

2. COMPACT SERIALIZATION:
   Instead of: {"id":1,"name":"Laptop Pro","description":"Premium laptop",
                "price":999.99,"category":"Electronics","imageUrl":"..."}
   Consider:   Remove unnecessary fields from cached objects
   Create a CacheDto with only essential fields

3. TTL MANAGEMENT:
   Hot data: 10 min TTL (frequently accessed)
   Warm data: 5 min TTL (moderately accessed)
   Cold data: DON'T CACHE (use DB directly)

4. KEY CLEANUP:
   redis-cli --scan --pattern "stale:*" | xargs redis-cli DEL
   Use SCAN (not KEYS) in production → non-blocking
```

9.6 Benchmarking Commands

```
# Redis built-in benchmark
redis-benchmark -c 100 -n 100000 -q
# SET: 112359.55 requests/sec
# GET: 121654.50 requests/sec

# Benchmark specific commands
redis-benchmark -c 50 -n 100000 -t set,get,incr -q
# SET: 110375.28 requests/sec
# GET: 116279.07 requests/sec
# INCR: 113636.36 requests/sec

# Benchmark with pipeline
```

```
redis-benchmark -c 50 -n 100000 -P 16 -t set,get -q
# SET: 892857.14 requests/sec  (8x with pipelining!)
# GET: 1041666.75 requests/sec

# Check latency
redis-cli --latency
# min: 0, max: 1, avg: 0.15 (milliseconds)

# Memory analysis
redis-cli INFO memory
redis-cli --bigkeys           # Find largest keys
redis-cli MEMORY DOCTOR       # Memory optimization advice
```

---

10. Security & Monitoring

10.1 Redis Authentication (AUTH)

```
# redis.conf — Basic authentication
requirepass YourStrongPassword123!

# Spring Boot configuration
spring:
  data:
    redis:
      password: ${REDIS_PASSWORD}
```

10.2 Redis ACLs (6.0+)

```
# Fine-grained access control per service

# Product Service: can GET/SET on product:* keys only
user product-service on >productServicePass ~product:* +get +set +del +mget +mset

# Inventory Service: can SET/GET/DEL on inventory:* keys only
user inventory-service on >inventoryServicePass ~inventory:* +get +set +del +setnx +expire

# API Gateway: can INCR/GET on ratelimit:* keys only
user api-gateway on >gatewayPass ~ratelimit:* +get +incr +expire +ttl

# Disable default user (security best practice)
user default off

# Apply ACLs
redis-cli ACL SETUSER product-service on >productServicePass ~product:* +get +set +del
redis-cli ACL SAVE   # Persist ACLs to disk
```

10.3 SSL/TLS Encryption

```yaml
# Spring Boot Redis with TLS
spring:
  data:
    redis:
      ssl:
        enabled: true
      host: redis.internal.company.com
      port: 6380            # TLS port (convention)
```

```
# redis.conf — TLS configuration
tls-port 6380
tls-cert-file /etc/redis/tls/redis.crt
tls-key-file /etc/redis/tls/redis.key
tls-ca-cert-file /etc/redis/tls/ca.crt
tls-auth-clients yes       # Require client certificates
port 0                     # Disable non-TLS port
```

10.4 Redis Sentinel (High Availability)

```
SENTINEL ARCHITECTURE:

  ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
  │  Sentinel 1  │     │  Sentinel 2  │     │  Sentinel 3  │
  │  (monitor)   │     │  (monitor)   │     │  (monitor)   │
  └──────┬───────┘     └──────┬───────┘     └──────┬───────┘
         │                    │                    │
         ▼                    ▼                    ▼
  ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
  │   MASTER     │────▶│  REPLICA 1   │     │  REPLICA 2   │
  │  (read/write)│     │  (read only) │     │  (read only) │
  └──────────────┘     └──────────────┘     └──────────────┘

FAILOVER PROCESS:
  1. Sentinel detects master is down (quorum vote)
  2. Sentinel promotes a replica to master
  3. Other replicas reconfigure to follow new master
  4. Application reconnects automatically (via Spring Data Redis)
  5. Total failover time: ~15-30 seconds

SPRING BOOT SENTINEL CONFIGURATION:
  spring:
    data:
      redis:
        sentinel:
          master: mymaster
          nodes:
            - sentinel-1:26379
            - sentinel-2:26379
```

```
            - sentinel-3:26379
          password: ${REDIS_PASSWORD}
```

10.5 Monitoring Stack

```
CRITICAL METRICS TO MONITOR:

1. MEMORY USAGE:
   redis-cli INFO memory
   → used_memory_human, maxmemory_human, mem_fragmentation_ratio
   ALERT: used_memory > 80% of maxmemory

2. CONNECTED CLIENTS:
   redis-cli INFO clients
   → connected_clients, blocked_clients
   ALERT: connected_clients > max_active_pool_size × services

3. KEY STATISTICS:
   redis-cli INFO keyspace
   → db0:keys=1523,expires=1500
   ALERT: keys growing without corresponding expires

4. HIT RATE:
   redis-cli INFO stats
   → keyspace_hits, keyspace_misses
   → Hit rate = hits / (hits + misses) × 100%
   TARGET: > 90% hit rate
   ALERT: < 70% hit rate → cache not effective

5. SLOW LOG:
   redis-cli SLOWLOG GET 10
   → Commands taking > 10ms (configurable)
   ALERT: Frequent slow commands → optimize queries

6. REPLICATION LAG:
   redis-cli INFO replication
   → master_repl_offset, slave_repl_offset
   ALERT: Lag > 1000 bytes → replica falling behind

PROMETHEUS INTEGRATION:
  # Deploy redis-exporter as sidecar
  redis-exporter:
    image: oliver006/redis_exporter:latest
    environment:
      REDIS_ADDR: redis://redis:6379
    ports:
      - "9121:9121"

  # Prometheus scrape config
  - job_name: redis
    static_configs:
```

```
      - targets: ['redis-exporter:9121']

GRAFANA DASHBOARDS:
  - Redis Overview (memory, connections, hit rate)
  - Redis Commands (throughput per command type)
  - Redis Keys (key count, expiry trends)
```

10.6 Alerting Rules

```yaml
# Prometheus alerting rules for Redis

groups:
  - name: redis-alerts
    rules:
      - alert: RedisHighMemoryUsage
        expr: redis_memory_used_bytes / redis_memory_max_bytes > 0.8
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Redis memory usage > 80%"

      - alert: RedisTooManyConnections
        expr: redis_connected_clients > 200
        for: 5m
        labels:
          severity: warning

      - alert: RedisLowHitRate
        expr: (redis_keyspace_hits_total / (redis_keyspace_hits_total +
redis_keyspace_misses_total)) < 0.7
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Redis cache hit rate below 70%"

      - alert: RedisDown
        expr: redis_up == 0
        for: 1m
        labels:
          severity: critical
```

---

11. Interview Preparation Questions

11.1 Core Redis Questions

Q1: What is Redis and why did you use it in your project?

Redis is an in-memory data structure store used as a cache, database, and message broker. We use it in three ways: (1) **Application caching** in Product Service — `@Cacheable` with `RedisCacheManager` stores product data in Redis with 10-minute TTL, reducing MySQL queries by ~90%. (2) **Distributed locking** in Inventory Service — `SETNX` with TTL prevents race conditions when multiple instances simultaneously try to reserve the same stock. (3) **Rate limiting** in API Gateway — Redis-backed counters limit each user to 100 requests per minute across all gateway instances.

Q2: Why Redis over Memcached?

Memcached is a simpler, multi-threaded cache supporting only key-value strings. Redis offers: rich data structures (strings, hashes, lists, sets, sorted sets, streams), persistence (RDB/AOF), distributed locking (SETNX with TTL), pub/sub messaging, replication, clustering, Lua scripting, and transactions. For our project, we needed distributed locking (SETNX) and JSON serialization — both impossible with Memcached.

Q3: Explain how @Cacheable works internally.

`@Cacheable` uses Spring AOP to create a proxy around the service method. When `getProductById(1)` is called: (1) The proxy intercepts the call. (2) It generates the cache key: `product::1`. (3) It calls `CacheManager.getCache("product").get("1")`. (4) If Redis returns a value → deserialize JSON → return directly (method body NEVER executes). (5) If Redis returns nil → call the actual method → serialize result → store in Redis with TTL → return to caller. The `"CACHE MISS"` log only appears when the method body actually executes.

Q4: How does your distributed lock prevent overselling?

We use Redis SETNX (Set if Not Exists) with a 30-second TTL. When reserving stock: (1) Generate a unique UUID as lock value. (2) Execute `SET inventory:lock:{productId} {uuid} NX EX 30`. (3) If key already exists → another thread holds the lock → return false. (4) If set succeeds → enter critical section → read stock → update stock → release lock. (5) Lock release checks that our UUID matches the stored value (prevents releasing another thread's lock). We also have Layer 2 protection: JPA `@Version` (optimistic locking) catches edge cases where Redis lock fails due to network issues.

Q5: What is the Cache-Aside pattern and why did you choose it?

Cache-Aside (Lazy Loading): the application manages the cache explicitly. On read: check cache → miss → query DB → populate cache. On write: update DB → evict cache. We chose it because: (1) Only frequently accessed data is cached (efficient memory usage). (2) Cache failure doesn't prevent reads (fallback to DB). (3) Spring's `@Cacheable` / `@CacheEvict` implements it natively. Alternative: Write-Through (cache always has latest data, but every write touches cache even if data is never read).

Q6: How do you handle cache invalidation?

Three mechanisms: (1) `@CacheEvict` on write operations — `createProduct()` evicts the product list cache; `deleteProduct()` evicts both individual and list caches. (2) `@CachePut` on updates — `updateProduct()` updates the specific product cache entry AND evicts the list cache. (3) TTL-based expiry — even if eviction fails, cache expires after 10 minutes (products) or 5 minutes (lists). This three-layer approach ensures eventual consistency within 10 minutes maximum.

Q7: What is the Cache Stampede problem and how do you prevent it?

Cache Stampede (Thundering Herd): a popular key expires → hundreds of concurrent requests all experience cache miss → all query the database simultaneously → database overwhelmed. Prevention strategies: (1) `sync=true` on `@Cacheable` — Spring acquires a mutex, only one thread queries DB, others wait. (2) Staggered TTL: add random jitter (TTL = 10min + random(0-2min)) so keys expire at different times. (3) Background refresh: proactively refresh hot keys before they expire.

Q8: Explain the difference between @Cacheable, @CachePut, and @CacheEvict.

`@Cacheable` : checks cache BEFORE method → returns cached value on hit → only calls method on miss. Use for reads. `@CachePut` : ALWAYS calls the method → updates cache with return value. Use for updates (execute business logic AND refresh cache). `@CacheEvict` : calls the method → removes cache entries AFTER. Use for deletes/creates (stale entries must be removed). Example: `updateProduct()` uses both `@CachePut` (update the specific product) and `@CacheEvict` (clear the list).

11.2 Architecture Questions

Q9: How would you design a multi-tier caching system?

L1 (In-Process): Caffeine with 1-minute TTL → zero network latency, handles 90% of reads. L2 (Distributed): Redis with 10-minute TTL → shared across all instances, handles L1 misses. L3 (Database): MySQL → handles L2 misses. Read flow: L1 → L2 → L3. Write flow: update DB → evict L2 → invalidate L1 (via distributed notification or TTL). Benefits: 99% of reads never leave the JVM, Redis handles cross-instance consistency.

Q10: What happens when Redis goes down in your system?

Product Service (caching): Every `@Cacheable` call becomes a cache miss → falls back to MySQL. Performance degrades (5ms → 10ms per request) but the service remains operational. Inventory Service (locking): `setIfAbsent()` throws `RedisConnectionFailureException` → caught in try-catch → `reserveStock()` returns false → order is not processed. This is actually SAFE — we'd rather reject orders than risk overselling. API Gateway (rate limiting): Rate limiter fails open (or closed, depending on configuration) — typically, we configure fail-open to maintain availability.

Q11: How do you handle Redis key expiration in a high-traffic system?

Redis doesn't delete expired keys immediately. It uses two strategies: (1) Lazy expiration: key checked on access → if expired → deleted → return nil. (2) Active expiration: background thread samples random keys → deletes expired ones (~20 keys/100ms). For high-traffic systems: set `maxmemory-policy` to `allkeys-lru` → even if expired keys aren't cleaned up, LRU eviction prevents memory overflow. Monitor `expired_keys` metric to ensure cleanup keeps pace.

Q12: What is the difference between Redis persistence options (RDB vs AOF)?

RDB: periodic point-in-time snapshots (binary dump). Fast recovery, compact files, but potential data loss between snapshots (minutes). AOF: logs every write command. Minimal data loss (1 second with `appendfsync everysec`), but larger files and slower recovery (replays all commands). Production recommendation: both RDB + AOF. RDB for fast recovery, AOF for minimal data loss. For caching use case (our project): persistence is optional — cache data is ephemeral and can be rebuilt from DB.

Q13: How would you scale Redis for 10x traffic?

1. Redis Cluster: shard data across 6+ master nodes (automatic data distribution via hash slots). 2. Read Replicas: route reads to replicas, writes to masters. 3. Connection pooling: increase `max-active` proportionally. 4. L1 cache (Caffeine): absorb 90% of reads in-process. 5. Pipeline batch operations: reduce network round-trips. 6. Compress large values: Snappy/LZ4 before caching. 7. Review key design: ensure even distribution across cluster nodes (no hot slots).

Q14: What are Redis Lua scripts and when would you use them?

Lua scripts execute atomically on the Redis server — multiple commands run as a single operation without interruption. Use cases: (1) Atomic rate limiting — INCR + EXPIRE as one operation (prevents race between increment and TTL set). (2) Safe lock release — GET + compare + DEL atomically (our project's UUID check could be a Lua script for perfect safety). (3) Complex aggregations — compute results server-side, return only the result (reduces network transfer). Lua scripts are cached by Redis (SHA1 hash) for repeated execution without retransmission.

12. Future Scalability Considerations

12.1 Redis Cluster Deployment

```
CURRENT: Single Redis node (development)
FUTURE:  6-node Redis Cluster (3 masters + 3 replicas)

ARCHITECTURE:
  Master A (slots 0-5460)    → Replica A1
  Master B (slots 5461-10922) → Replica B1
  Master C (slots 10923-16383) → Replica C1

SPRING BOOT CONFIGURATION:
  spring:
    data:
      redis:
        cluster:
          nodes:
            - redis-node-1:6379
            - redis-node-2:6379
            - redis-node-3:6379
          max-redirects: 3

BENEFITS:
  - Automatic data sharding (no application changes)
  - Linear scalability (add nodes → more capacity)
  - Automatic failover (replica promoted on master failure)
```

12.2 Multi-Tier Caching (Caffeine + Redis)

```
ARCHITECTURE:
  Request → L1 (Caffeine, in-JVM, 1 min) → L2 (Redis, network, 10 min) → MySQL

SPRING BOOT IMPLEMENTATION:
  @Bean
  public CacheManager cacheManager(RedisConnectionFactory redisFactory) {
      CaffeineCacheManager l1 = new CaffeineCacheManager();
      l1.setCaffeine(Caffeine.newBuilder()
          .maximumSize(1000)
          .expireAfterWrite(Duration.ofMinutes(1)));

      RedisCacheManager l2 = RedisCacheManager.builder(redisFactory)
          .cacheDefaults(config.entryTtl(Duration.ofMinutes(10)))
          .build();

      return new CompositeCacheManager(l1, l2);
  }

PERFORMANCE:
  L1 hit: ~0.001ms (in-memory, no network)
```

```
  L2 hit: ~0.5ms (Redis, network)
  L3 (DB): ~5ms

  Expected hit rates:
    L1: 85% → 85% of requests served in ~0.001ms
    L2: 10% → 10% of requests served in ~0.5ms
    DB:  5% → 5% of requests served in ~5ms
    Weighted average: ~0.26ms per request
```

12.3 Redis Streams as Kafka Alternative

```
FOR LOW-VOLUME USE CASES:
  Instead of deploying Kafka (operational overhead), use Redis Streams for:
  - Notification delivery
  - Audit logging
  - Activity feeds

SPRING BOOT REDIS STREAM CONSUMER:
  @Bean
  public StreamMessageListenerContainer<String, MapRecord<String, String, String>>
      listenerContainer(RedisConnectionFactory factory) {
      return StreamMessageListenerContainer.create(factory,
          StreamMessageListenerContainerOptions.builder()
              .pollTimeout(Duration.ofMillis(100))
              .build());
  }
```

12.4 Session Management (Redis)

```
FUTURE ENHANCEMENT: Centralized Session Store

CURRENT: JWT is stateless (no session store needed)
FUTURE:  Redis-backed sessions for refresh tokens, active sessions

SPRING SESSION:
  <dependency>
      <groupId>org.springframework.session</groupId>
      <artifactId>spring-session-data-redis</artifactId>
  </dependency>

  spring:
    session:
      store-type: redis
      redis:
        flush-mode: on-save
        namespace: spring:session

BENEFITS:
  - Centralized session store across all instances
  - Session survives server restart (Redis persistence)
```

```
  - Easy session invalidation (force logout)
  - Horizontal scaling (any instance can serve any user)
```

12.5 Kubernetes Deployment

```
REDIS ON KUBERNETES:

OPTION 1: Bitnami Redis Helm Chart
  helm install redis bitnami/redis \
    --set architecture=replication \
    --set replica.replicaCount=3 \
    --set auth.password=${REDIS_PASSWORD}

OPTION 2: Redis Operator (redis-operator)
  Manages Redis Cluster lifecycle on K8s
  Auto-scaling, auto-failover, auto-backup

PERSISTENT STORAGE:
  volumeClaimTemplates:
    - metadata:
        name: redis-data
      spec:
        storageClassName: gp2
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 10Gi
```

Appendix A — Redis Command Reference

Command	Usage	This Project
`SET key value`	Store a string value	Product cache (via Spring)
`GET key`	Retrieve a string value	Product cache (via Spring)
`SET key value NX EX 30`	Set if not exists with 30s TTL	Distributed lock (SETNX)
`DEL key`	Delete a key	Lock release, cache evict
`INCR key`	Atomic increment	Rate limiting counter
`EXPIRE key seconds`	Set TTL on existing key	Rate limit window
`TTL key`	Check remaining TTL	Debugging
`KEYS pattern`	Find keys by pattern (SLOW!)	Debugging only
`SCAN cursor MATCH pattern`	Iterate keys safely	Production key enumeration
`INFO section`	Server statistics	Monitoring
`DBSIZE`	Total key count	Monitoring

`FLUSHDB`	Delete ALL keys in current DB	Testing only!
`MONITOR`	Real-time command stream	Debugging
`SLOWLOG GET 10`	Slow command log	Performance tuning

Appendix B — Glossary

Term	Definition
Redis	Remote Dictionary Server — in-memory data structure store
SETNX	SET if Not eXists — atomic lock primitive
TTL	Time To Live — automatic key expiration
RDB	Redis Database — point-in-time binary snapshot
AOF	Append-Only File — write-ahead log for durability
Cache-Aside	Pattern where application manages cache population
Write-Through	Pattern where every write updates cache AND database
Write-Behind	Pattern where writes go to cache first, DB async later
Cache Stampede	Many concurrent cache misses for the same key
Cache Penetration	Queries for non-existent data bypass cache
Cache Avalanche	Mass key expiration causes DB overload
Hot Key	Single key receiving disproportionately high traffic
Eviction	Removing keys when Redis reaches maxmemory
LRU	Least Recently Used — eviction based on last access time
LFU	Least Frequently Used — eviction based on access frequency
Sentinel	Redis high-availability solution (monitoring + failover)
Cluster	Redis horizontal scaling (data sharding across nodes)
Hash Slot	Unit of data distribution in Redis Cluster (0-16383)
RedLock	Distributed lock algorithm across multiple Redis instances
Pipeline	Batch multiple Redis commands in one network round-trip
Lua Script	Server-side script for atomic multi-command execution
StringRedisTemplate	Spring template for String-based Redis operations
RedisCacheManager	Spring CacheManager backed by Redis
GenericJackson2JsonRedisSerializer	JSON serializer for cache values

---

End of Document

This guide provides a comprehensive, enterprise-level reference for designing and implementing Redis-backed architecture in Spring Boot Microservices. For implementation, start with Step 1 (Redis Setup), then proceed through caching (Step 2), distributed locking (Step 3), rate limiting (Step 4), and pub/sub (Step 5). Each section is self-contained and can be referenced independently.